

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №8
по дисциплине «Вычислительная математика»
Тема: Решение систем линейных уравнений методом Якоби.

Студент гр. 4342

Митюков М.Н.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Разработать программу для решения системы линейных уравнений методом Якоби (методом простых итераций), реализовать выбор ведущего элемента, проверку существования решения и невырожденности матрицы в процессе вычислений, а также исследовать работу программы на базовых, больших и специальных матрицах, включая матрицы Гильберта, с оценкой корректности получаемого решения, затрат по времени выполнения и используемой памяти.

Теоретический минимум.

Метод Якоби, или метод простых итераций для систем линейных уравнений, применяется для приближённого решения системы вида $A * x = b$, где A — квадратная матрица коэффициентов порядка n , x — вектор неизвестных, b — вектор правых частей. В отличие от прямых методов, которые стремятся получить точное решение за конечное число преобразований, метод Якоби строит последовательность приближений $x^{(0)}$, $x^{(1)}$, $x^{(2)}$, ..., которая при выполнении условий сходимости стремится к точному решению системы.

Для построения метода исходную матрицу A представляют в виде суммы трёх матриц: $A = D + L + U$, где D — диагональная часть матрицы, L — нижняя треугольная часть без диагонали, U — верхняя треугольная часть без диагонали. Тогда систему $A * x = b$ можно переписать в виде $D * x = b - (L + U) * x$. На этой основе получают итерационную формулу метода Якоби: $x^{(k+1)} = D^{(-1)} * (b - (L + U) * x^{(k)})$.

В покомпонентной форме формула метода записывается так: $x_i^{(k+1)} = (b_i - \sum_{j \neq i} a_{ij} * x_j^{(k)}) / a_{ii}$, где $i = 1, 2, \dots, n$. Это означает, что каждое новое приближение для i -й неизвестной вычисляется через значения всех остальных неизвестных с предыдущей итерации. Именно в этом состоит характерная особенность метода Якоби: на каждом шаге используются только значения предыдущего приближения, а не уже обновлённые компоненты текущего шага.

Работа метода начинается с выбора начального приближения $x^{(0)}$. После этого по итерационной формуле последовательно вычисляются векторы $x^{(1)}$, $x^{(2)}$, $x^{(3)}$ и так далее. Если последовательность приближений сходится, то её предел и является решением исходной системы. Таким образом, задача метода заключается не в непосредственном нахождении точного ответа, а в последовательном уточнении приближённого решения.

Для применимости метода необходимо, чтобы все диагональные элементы матрицы были отличны от нуля, поскольку в формуле присутствует деление на a_{ii} . Если хотя бы один диагональный элемент равен нулю, то соответствующая

формула теряет смысл. Поэтому важным условием использования метода является возможность привести систему к виду, в котором диагональные элементы ненулевые.

Сходимость метода Якоби определяется свойствами итерационной матрицы $B = -D^{-1} * (L + U)$. Метод сходится тогда и только тогда, когда спектральный радиус этой матрицы меньше единицы, то есть $\rho(B) < 1$. Это условие является точным теоретическим критерием сходимости. Однако на практике чаще пользуются более простыми достаточными условиями. Наиболее известное из них состоит в строгом диагональном преобладании матрицы по строкам: $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ для каждой строки i . Если это условие выполняется, то метод Якоби гарантированно сходится.

Если строгого диагонального преобладания нет, это ещё не означает, что метод обязательно расходится. В отдельных случаях он может сходиться и без этого условия, однако гарантии уже нет. При нарушении условий сходимости последовательность приближений может сходиться очень медленно, колебаться или вообще расходиться. Поэтому вопрос сходимости является одним из центральных при изучении метода Якоби.

Для оценки процесса приближений вводят критерий останова. Обычно сравнивают соседние итерации и проверяют, стало ли различие между ними достаточно малым. Также можно использовать невязку $r^{(k)} = A * x^{(k)} - b$, которая показывает, насколько текущее приближение удовлетворяет исходной системе. Чем меньше невязка, тем ближе найденный вектор к точному решению. Таким образом, метод Якоби позволяет не только строить приближения, но и контролировать качество получаемого результата.

С точки зрения вычислительной математики метод Якоби относится к простым и наглядным итерационным методам. Его основными достоинствами являются простота вывода, понятная структура вычислений и возможность последовательного улучшения приближённого решения. В то же время скорость его сходимости существенно зависит от свойств матрицы системы. Поэтому метод Якоби особенно важен как базовый итерационный метод, на котором

удобно изучать общие принципы численного решения систем линейных уравнений.

Выполнение работы.

В ходе выполнения работы была разработана программа на языке C++, предназначенная для решения системы линейных уравнений методом Якоби, или методом простых итераций. При подготовке системы к вычислениям был реализован выбор ведущего элемента. В качестве стратегии использован частичный выбор по столбцу: для каждого столбца среди доступных строк находится элемент с наибольшим модулем, после чего при необходимости выполняется перестановка строк. Такое преобразование позволяет избежать появления нулевых и слишком малых диагональных элементов, а также делает последующий итерационный процесс более устойчивым.

На начальном этапе работы программы выполняется проверка исходных данных. Контролируется, что матрица коэффициентов является квадратной, а размер вектора правых частей соответствует числу уравнений системы. После этого система подготавливается к дальнейшему анализу и вычислениям. Если входные данные не удовлетворяют необходимым условиям, программа завершает работу с сообщением об ошибке, не переходя к итерационному процессу.

Основу решения составляет итерационная схема метода Якоби. Для каждой неизвестной на очередном шаге вычисляется новое приближение по значениям предыдущей итерации. Таким образом, на шаге $k+1$ все компоненты нового вектора решения определяются только через значения шага k . После вычисления очередного приближения производится оценка разности между соседними итерациями. Если эта разность становится меньше заданной точности, итерационный процесс прекращается, и текущее приближение принимается за решение. Дополнительно вводится ограничение на максимальное число итераций, что позволяет корректно завершать работу программы в случае отсутствия сходимости или слишком медленного приближения к решению.

В программе также реализована проверка существования решения и контроль невырожденности матрицы. До начала итераций выполняется анализ

системы на совместность. Для этого используется ранговый подход: сравниваются ранг матрицы коэффициентов A и ранг расширенной матрицы $[A|b]$. Если $\text{rank}(A) \neq \text{rank}([A|b])$, система признаётся несовместной, и программа сообщает об отсутствии решения. Если $\text{rank}(A) = \text{rank}([A|b])$, но меньше числа неизвестных, делается вывод о наличии бесконечно большого числа решений. Если же $\text{rank}(A) = \text{rank}([A|b]) = n$, система имеет единственное решение. Тем самым ещё до запуска итераций определяется, существует ли решение вообще и можно ли рассчитывать на получение единственного ответа. Одновременно с этим в ходе подготовки системы и вычислений контролируется наличие ненулевых диагональных элементов, что позволяет выявлять вырожденные или некорректно преобразованные случаи.

Структура программы была организована таким образом, чтобы разделить основные части решения по функциональному назначению. В отдельном модуле реализованы структуры данных, численные проверки, выбор ведущего элемента, вычисление рангов, итерационный алгоритм Якоби и вспомогательные функции для анализа результата. В файле `main.cpp` организован пользовательский интерфейс командной строки, обеспечивающий запуск программы, чтение системы из входных данных или из файла, а также вывод результата. Отдельно был создан файл `test.cpp`, содержащий набор автоматических тестов, предназначенных для проверки корректности работы программы на различных типах матриц.

Для удобства тестирования в программу были добавлены средства генерации матриц различных типов. Отдельно реализована генерация матриц Гильберта, которые используются как специальные плохо обусловленные примеры. Кроме того, реализована генерация больших матриц нескольких видов: плотных диагонально преобладающих, трёхдиагональных, пятидиагональных, ленточных и симметричных ленточных матриц. Это позволило проводить испытания не только на простых учебных системах малого размера, но и на существенно более крупных примерах, характерных для численных методов.

В процессе тестирования сначала были рассмотрены базовые случаи с небольшими матрицами, для которых заранее известно точное решение. Это позволило проверить правильность вычислений, корректность остановки по заданной точности и правильность оценки найденного ответа. Затем были проверены несовместные и вырожденные системы, где программа должна была не выдавать численный результат, а корректно определять отсутствие решения или наличие бесконечного числа решений. Эти испытания подтвердили правильность логики предварительного анализа системы.

Отдельное внимание было уделено специальным матрицам Гильберта. Такие матрицы являются классическим примером плохо обусловленных систем, на которых численные методы могут работать нестабильно или сходиться медленно. Тестирование на этих примерах проводилось для того, чтобы убедиться, что программа корректно реагирует на трудные случаи, не выдаёт ложного результата и корректно фиксирует отсутствие сходимости при неблагоприятных условиях.

Кроме того, были выполнены эксперименты на матрицах больших размеров, в том числе порядка 1000×1000 и 10000×10000 . Для этих целей использовались различные структурированные генераторы матриц, позволяющие исследовать работу метода Якоби на более реалистичных примерах. В ходе испытаний оценивались не только правильность получаемого решения, но и время выполнения программы, а также объём используемой памяти. Это позволило проанализировать применимость метода к большим системам и сравнить его поведение на различных типах матриц.

В результате выполнения работы была создана программа, реализующая решение системы линейных уравнений методом Якоби с выбором ведущего элемента, проверкой существования решения, контролем невырожденности матрицы и расширенным набором тестов для базовых, специальных и больших примеров. Полученная программа соответствует поставленной цели работы и позволяет исследовать особенности применения метода Якоби на различных классах систем линейных уравнений.

Тестирование.

Код тестов приведен в Приложении А. Тестирование приведено в таблице 1 в Приложении Б.

Вывод.

В ходе выполнения работы был реализован и исследован метод Якоби, или метод простых итераций, для решения систем линейных уравнений. В программе был предусмотрен выбор ведущего элемента по столбцу, что позволило повысить корректность подготовки системы к вычислениям и избежать проблем, связанных с нулевыми или слишком малыми диагональными элементами. Также были реализованы проверки существования решения, совместности системы и невырожденности матрицы, благодаря чему программа способна не только находить решение, но и корректно определять случаи, когда система несовместна или имеет бесконечно много решений.

Проведённое тестирование включало базовые примеры, специальные матрицы Гильберта и матрицы больших размеров различных типов. Полученные результаты показали, что реализованный алгоритм корректно работает на системах, удовлетворяющих условиям сходимости метода Якоби, и позволяет получать приближённые решения с заданной точностью. Проверка на специальных и трудных примерах подтвердила, что программа корректно обрабатывает случаи неблагоприятного поведения метода и не выдаёт ложного результата при отсутствии устойчивой сходимости.

Дополнительно в ходе работы были получены практические оценки времени выполнения и объёма используемой памяти, что позволило проанализировать вычислительную эффективность метода на больших системах. В результате можно сделать вывод, что разработанная программа соответствует поставленным требованиям, корректно реализует метод Якоби и может использоваться как для учебного исследования итерационных методов решения СЛУ, так и для практического анализа поведения метода на различных классах матриц.

ПРИЛОЖЕНИЕ А

Название файла: jacobi_solver.hpp

```
#ifndef JACOBI_SOLVER_HPP
#define JACOBI_SOLVER_HPP

#include <string>
#include <vector>

namespace jacobi {

using Matrix = std::vector<std::vector<double>>>;
using Vector = std::vector<double>;

// Тип решения системы.
enum class SolutionType {
    UNIQUE,      // Единственное решение.
    NONE,        // Решений нет.
    INFINITE     // Решений бесконечно много.
};

// Информация о системе: ранги, тип решения и признак невырожденности.
struct SystemStatus {
    SolutionType type = SolutionType::NONE;
    int rank_a = 0;
    int rank_augmented = 0;
    bool nonsingular = false;
};

// Результат работы метода Якоби.
struct JacobiResult {
    bool converged = false;           // Сошёлся ли метод.
    bool guaranteed_convergence = false; // Есть ли достаточное условие
сходимости.
    int iterations = 0;              // Число итераций.
    double final_error = 0.0;        // ||x(k+1) - x(k)||_inf.
    double residual_norm = 0.0;      // ||Ax - b||_inf.
    Vector x;                        // Найденное приближение.
    std::string message;             // Пояснение для пользователя.
};

// Специальное хранение большой трёхдиагональной системы.
// Такой формат очень экономичен по памяти.
struct TridiagonalSystem {
    Vector lower; // Поддиагональ, размер n-1.
    Vector diag;  // Главная диагональ, размер n.
    Vector upper; // Наддиагональ, размер n-1.
    Vector b;     // Правая часть.
};

// Универсальное хранение ленточной матрицы.
// Матрица хранится не полностью, а только по диагоналям.
// diagonals[index][i] = a[i][i + offset], где
// offset = index - lower_band.
struct BandedSystem {
    int lower_band = 0; // Сколько диагоналей ниже главной
хранится.
};

}
```

```

    int upper_band = 0;                // Сколько диагоналей выше главной
хранится.
    std::vector<Vector> diagonals;     // Всего lower_band + upper_band + 1
диагоналей.
    Vector b;                          // Правая часть.
};

// ----- Базовые сервисные функции -----

double vectorInfinityNorm(const Vector& v);
double residualInfinityNorm(const Matrix& a, const Vector& x, const
Vector& b);
double residualInfinityNorm(const TridiagonalSystem& s, const Vector& x);
double residualInfinityNorm(const BandedSystem& s, const Vector& x);

// ----- Анализ системы -----

SystemStatus analyzeSystem(const Matrix& a, const Vector& b, double eps =
1e-12);
bool reorderRowsByLeadingElement(Matrix& a, Vector& b, double eps = 1e-
12);
bool isStrictlyDiagonallyDominant(const Matrix& a, double eps = 1e-12);
bool isStrictlyDiagonallyDominant(const TridiagonalSystem& s, double eps
= 1e-12);
bool isStrictlyDiagonallyDominant(const BandedSystem& s, double eps = 1e-
12);
double iterationMatrixInfinityNorm(const Matrix& a, double eps = 1e-12);

// ----- Решение методом Якоби -----

JacobiResult solveJacobi(
    Matrix a,
    Vector b,
    double eps = 1e-10,
    int max_iterations = 10000,
    bool try_reorder = true
);

JacobiResult solveJacobi(
    const TridiagonalSystem& s,
    double eps = 1e-10,
    int max_iterations = 100000
);

JacobiResult solveJacobi(
    const BandedSystem& s,
    double eps = 1e-10,
    int max_iterations = 100000
);

// ----- Генераторы тестовых систем -----

Matrix generateHilbertMatrix(int n);
void generateHilbertSystem(int n, Matrix& a, Vector& b, const Vector&
exact_x);

// Плотная строго диагонально доминирующая матрица.
void generateDenseDiagonallyDominantSystem(

```

```

    int n,
    Matrix& a,
    Vector& b,
    const Vector& exact_x,
    unsigned seed = 123456789u
);

// Большая трёхдиагональная система.
TridiagonalSystem generateLargeTridiagonalSystem(int n, const Vector&
exact_x);

// Большая пятидиагональная система.
BandedSystem generatePentadiagonalSystem(
    int n,
    const Vector& exact_x,
    unsigned seed = 123456789u
);

// Общая большая ленточная система.
BandedSystem generateBandedDiagonallyDominantSystem(
    int n,
    int lower_band,
    int upper_band,
    const Vector& exact_x,
    unsigned seed = 123456789u
);

// Симметричная положительно определённая ленточная система.
BandedSystem generateSymmetricPositiveDefiniteBandedSystem(
    int n,
    int half_band,
    const Vector& exact_x,
    unsigned seed = 123456789u
);

// ----- Работа с файлами -----

void saveDenseSystemToFile(
    const Matrix& a,
    const Vector& b,
    const std::string& filename,
    double eps,
    int max_iterations
);

void saveTridiagonalSystemToFile(
    const TridiagonalSystem& s,
    const std::string& filename,
    double eps,
    int max_iterations
);

void saveBandedSystemToFile(
    const BandedSystem& s,
    const std::string& filename,
    double eps,
    int max_iterations
);

```

```
} // namespace jacobi
```

```
#endif
```

Название файла: jacobi_solver.cpp

```
#include "jacobi_solver.hpp"

#include <algorithm>
#include <cmath>
#include <fstream>
#include <limits>
#include <numeric>
#include <random>
#include <stdexcept>

namespace jacobi {

namespace {

double absd(double x) {
    return std::fabs(x);
}

// Поиск ранга методом Гаусса с выбором главного элемента по столбцу.
int rankOfMatrix(Matrix m, double eps) {
    const int rows = static_cast<int>(m.size());
    const int cols = rows == 0 ? 0 : static_cast<int>(m[0].size());

    int rank = 0;
    int current_row = 0;

    for (int col = 0; col < cols && current_row < rows; ++col) {
        int pivot_row = current_row;
        for (int r = current_row + 1; r < rows; ++r) {
            if (absd(m[r][col]) > absd(m[pivot_row][col])) {
                pivot_row = r;
            }
        }

        if (absd(m[pivot_row][col]) <= eps) {
            continue;
        }

        std::swap(m[current_row], m[pivot_row]);

        for (int r = current_row + 1; r < rows; ++r) {
            const double factor = m[r][col] / m[current_row][col];
            if (absd(factor) <= eps) {
                continue;
            }
            for (int c = col; c < cols; ++c) {
                m[r][c] -= factor * m[current_row][c];
            }
        }

        ++rank;
    }
}
```

```

        ++current_row;
    }

    return rank;
}

Vector multiply(const Matrix& a, const Vector& x) {
    const int n = static_cast<int>(a.size());
    Vector result(n, 0.0);

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < static_cast<int>(a[i].size()); ++j) {
            result[i] += a[i][j] * x[j];
        }
    }

    return result;
}

// Возвращает значение из ленточной матрицы.
// Если элемент находится вне ленты, возвращается 0.
double getBandedValue(const BandedSystem& s, int row, int col) {
    const int n = static_cast<int>(s.b.size());
    if (row < 0 || row >= n || col < 0 || col >= n) {
        return 0.0;
    }

    const int offset = col - row;
    if (offset < -s.lower_band || offset > s.upper_band) {
        return 0.0;
    }

    const int index = offset + s.lower_band;
    return s.diagonals[index][row];
}

// Вычисление b = A * x для ленточной матрицы.
Vector multiply(const BandedSystem& s, const Vector& x) {
    const int n = static_cast<int>(s.b.size());
    Vector result(n, 0.0);

    for (int i = 0; i < n; ++i) {
        const int left = std::max(0, i - s.lower_band);
        const int right = std::min(n - 1, i + s.upper_band);

        for (int j = left; j <= right; ++j) {
            result[i] += getBandedValue(s, i, j) * x[j];
        }
    }

    return result;
}

// Создание пустой ленточной системы нужного размера.
BandedSystem makeEmptyBandedSystem(int n, int lower_band, int upper_band)
{
    if (n <= 0) {

```

```

        throw std::invalid_argument("Размер системы должен быть
положительным.");
    }
    if (lower_band < 0 || upper_band < 0) {
        throw std::invalid_argument("Ширина ленты не может быть
отрицательной.");
    }

    BandedSystem s;
    s.lower_band = lower_band;
    s.upper_band = upper_band;
    s.diagonals.assign(lower_band + upper_band + 1, Vector(n, 0.0));
    s.b.assign(n, 0.0);
    return s;
}

void setBandedValue(BandedSystem& s, int row, int col, double value) {
    const int offset = col - row;
    if (offset < -s.lower_band || offset > s.upper_band) {
        throw std::invalid_argument("Попытка записать элемент вне ленты
матрицы.");
    }
    s.diagonals[offset + s.lower_band][row] = value;
}

} // namespace

double vectorInfinityNorm(const Vector& v) {
    double norm = 0.0;
    for (double value : v) {
        norm = std::max(norm, absd(value));
    }
    return norm;
}

double residualInfinityNorm(const Matrix& a, const Vector& x, const
Vector& b) {
    Vector ax = multiply(a, x);
    for (std::size_t i = 0; i < ax.size(); ++i) {
        ax[i] -= b[i];
    }
    return vectorInfinityNorm(ax);
}

double residualInfinityNorm(const TridiagonalSystem& s, const Vector& x)
{
    const int n = static_cast<int>(s.diag.size());
    double norm = 0.0;

    for (int i = 0; i < n; ++i) {
        double value = s.diag[i] * x[i];
        if (i > 0) {
            value += s.lower[i - 1] * x[i - 1];
        }
        if (i + 1 < n) {
            value += s.upper[i] * x[i + 1];
        }
        norm = std::max(norm, absd(value - s.b[i]));
    }
}

```



```

    }

    return norm;
}

double residualInfinityNorm(const BandedSystem& s, const Vector& x) {
    Vector ax = multiply(s, x);
    for (std::size_t i = 0; i < ax.size(); ++i) {
        ax[i] -= s.b[i];
    }
    return vectorInfinityNorm(ax);
}

SystemStatus analyzeSystem(const Matrix& a, const Vector& b, double eps)
{
    SystemStatus status;

    const int n = static_cast<int>(a.size());
    if (n == 0) {
        status.type = SolutionType::INFINITE;
        status.rank_a = 0;
        status.rank_augmented = 0;
        status.nonsingular = false;
        return status;
    }

    for (const auto& row : a) {
        if (static_cast<int>(row.size()) != n) {
            throw std::invalid_argument("Матрица A должна быть
квадратной.");
        }
    }

    if (static_cast<int>(b.size()) != n) {
        throw std::invalid_argument("Размер вектора b должен совпадать с
размером матрицы A.");
    }

    Matrix augmented = a;
    for (int i = 0; i < n; ++i) {
        augmented[i].push_back(b[i]);
    }

    status.rank_a = rankOfMatrix(a, eps);
    status.rank_augmented = rankOfMatrix(augmented, eps);

    if (status.rank_a != status.rank_augmented) {
        status.type = SolutionType::NONE;
        status.nonsingular = false;
    } else if (status.rank_a < n) {
        status.type = SolutionType::INFINITE;
        status.nonsingular = false;
    } else {
        status.type = SolutionType::UNIQUE;
        status.nonsingular = true;
    }

    return status;
}

```

```

bool reorderRowsByLeadingElement(Matrix& a, Vector& b, double eps) {
    const int n = static_cast<int>(a.size());
    if (static_cast<int>(b.size()) != n) {
        throw std::invalid_argument("Размер вектора b не совпадает с
числом строк матрицы A.");
    }

    Matrix new_a(n, Vector(n, 0.0));
    Vector new_b(n, 0.0);
    std::vector<bool> used(n, false);

    for (int col = 0; col < n; ++col) {
        int best_row = -1;
        double best_abs = -1.0;

        for (int row = 0; row < n; ++row) {
            if (used[row]) {
                continue;
            }
            const double current_abs = absd(a[row][col]);
            if (current_abs > best_abs) {
                best_abs = current_abs;
                best_row = row;
            }
        }

        if (best_row == -1 || best_abs <= eps) {
            return false;
        }

        used[best_row] = true;
        new_a[col] = a[best_row];
        new_b[col] = b[best_row];
    }

    a = std::move(new_a);
    b = std::move(new_b);
    return true;
}

bool isStrictlyDiagonallyDominant(const Matrix& a, double eps) {
    const int n = static_cast<int>(a.size());
    for (int i = 0; i < n; ++i) {
        double off_diagonal_sum = 0.0;
        for (int j = 0; j < n; ++j) {
            if (i != j) {
                off_diagonal_sum += absd(a[i][j]);
            }
        }
        if (!(absd(a[i][i]) > off_diagonal_sum + eps)) {
            return false;
        }
    }
    return true;
}

```

```

bool isStrictlyDiagonallyDominant(const TridiagonalSystem& s, double eps)
{
    const int n = static_cast<int>(s.diag.size());
    for (int i = 0; i < n; ++i) {
        double sum = 0.0;
        if (i > 0) {
            sum += absd(s.lower[i - 1]);
        }
        if (i + 1 < n) {
            sum += absd(s.upper[i]);
        }
        if (!(absd(s.diag[i]) > sum + eps)) {
            return false;
        }
    }
    return true;
}

bool isStrictlyDiagonallyDominant(const BandedSystem& s, double eps) {
    const int n = static_cast<int>(s.b.size());
    for (int i = 0; i < n; ++i) {
        double sum = 0.0;
        const int left = std::max(0, i - s.lower_band);
        const int right = std::min(n - 1, i + s.upper_band);
        for (int j = left; j <= right; ++j) {
            if (i == j) {
                continue;
            }
            sum += absd(getBandedValue(s, i, j));
        }
        if (!(absd(getBandedValue(s, i, i)) > sum + eps)) {
            return false;
        }
    }
    return true;
}

double iterationMatrixInfinityNorm(const Matrix& a, double eps) {
    const int n = static_cast<int>(a.size());
    double norm = 0.0;

    for (int i = 0; i < n; ++i) {
        if (absd(a[i][i]) <= eps) {
            return std::numeric_limits<double>::infinity();
        }

        double row_sum = 0.0;
        for (int j = 0; j < n; ++j) {
            if (i != j) {
                row_sum += absd(a[i][j] / a[i][i]);
            }
        }
        norm = std::max(norm, row_sum);
    }

    return norm;
}

```

```

JacobiResult solveJacobi(Matrix a, Vector b, double eps, int
max_iterations, bool try_reorder) {
    JacobiResult result;

    const int n = static_cast<int>(a.size());
    if (n == 0) {
        result.converged = true;
        result.guaranteed_convergence = true;
        result.message = "Пустая система: решение считается найденным.";
        return result;
    }

    for (const auto& row : a) {
        if (static_cast<int>(row.size()) != n) {
            throw std::invalid_argument("Матрица A должна быть
квадратной.");
        }
    }
    if (static_cast<int>(b.size()) != n) {
        throw std::invalid_argument("Размер вектора b должен совпадать с
размером матрицы A.");
    }
    if (eps <= 0.0 || max_iterations <= 0) {
        throw std::invalid_argument("eps и max_iterations должны быть
положительными.");
    }

    const SystemStatus status = analyzeSystem(a, b, 1e-12);
    if (status.type == SolutionType::NONE) {
        result.message = "Система несовместна: решений нет.";
        return result;
    }
    if (status.type == SolutionType::INFINITE) {
        result.message = "Система имеет бесконечно много решений: метод
Якоби в таком виде не применяется.";
        return result;
    }

    if (try_reorder) {
        reorderRowsByLeadingElement(a, b, 1e-12);
    }

    result.guaranteed_convergence = isStrictlyDiagonallyDominant(a, 1e-
12);
    const double norm_b = iterationMatrixInfinityNorm(a, 1e-12);

    Vector x_old(n, 0.0);
    Vector x_new(n, 0.0);

    for (int iteration = 1; iteration <= max_iterations; ++iteration) {
        for (int i = 0; i < n; ++i) {
            if (absd(a[i][i]) <= 1e-12) {
                result.message = "На диагонали встретился нулевой
элемент, метод Якоби неприменим.";
                result.iterations = iteration - 1;
                return result;
            }
        }
    }
}

```

```

        double sum = 0.0;
        for (int j = 0; j < n; ++j) {
            if (j != i) {
                sum += a[i][j] * x_old[j];
            }
        }
        x_new[i] = (b[i] - sum) / a[i][i];
    }

    Vector diff(n, 0.0);
    for (int i = 0; i < n; ++i) {
        diff[i] = x_new[i] - x_old[i];
    }

    result.final_error = vectorInfinityNorm(diff);
    result.iterations = iteration;

    if (result.final_error < eps) {
        result.converged = true;
        result.x = x_new;
        result.residual_norm = residualInfinityNorm(a, result.x, b);

        if (result.guaranteed_convergence) {
            result.message = "Метод Якоби сошёлся. Достаточное
условие сходимости выполнено.";
        } else if (norm_b < 1.0) {
            result.message = "Метод Якоби сошёлся. Гарантия
установлена по норме итерационной матрицы.";
            result.guaranteed_convergence = true;
        } else {
            result.message = "Метод Якоби сошёлся, но строгой
априорной гарантии сходимости не было.";
        }
        return result;
    }

    x_old.swap(x_new);

    result.x = x_old;
    result.residual_norm = residualInfinityNorm(a, result.x, b);
    result.message = "Достигнуто максимальное число итераций, требуемая
точность не достигнута.";
    return result;
}

JacobiResult solveJacobi(const TridiagonalSystem& s, double eps, int
max_iterations) {
    JacobiResult result;

    const int n = static_cast<int>(s.diag.size());
    if (n == 0) {
        result.converged = true;
        result.guaranteed_convergence = true;
        result.message = "Пустая система: решение считается найденным.";
        return result;
    }
    if (static_cast<int>(s.b.size()) != n ||

```

```

        static_cast<int>(s.lower.size()) != std::max(0, n - 1) ||
        static_cast<int>(s.upper.size()) != std::max(0, n - 1)) {
            throw std::invalid_argument("Некорректные размеры
трёхдиагональной системы.");
        }
        if (eps <= 0.0 || max_iterations <= 0) {
            throw std::invalid_argument("eps и max_iterations должны быть
положительными.");
        }

        result.guaranteed_convergence = isStrictlyDiagonallyDominant(s, 1e-
12);

        Vector x_old(n, 0.0);
        Vector x_new(n, 0.0);

        for (int iteration = 1; iteration <= max_iterations; ++iteration) {
            for (int i = 0; i < n; ++i) {
                if (absd(s.diag[i]) <= 1e-12) {
                    result.message = "На главной диагонали встретился нулевой
элемент, метод Якоби неприменим.";
                    result.iterations = iteration - 1;
                    return result;
                }

                double sum = 0.0;
                if (i > 0) {
                    sum += s.lower[i - 1] * x_old[i - 1];
                }
                if (i + 1 < n) {
                    sum += s.upper[i] * x_old[i + 1];
                }
                x_new[i] = (s.b[i] - sum) / s.diag[i];
            }

            Vector diff(n, 0.0);
            for (int i = 0; i < n; ++i) {
                diff[i] = x_new[i] - x_old[i];
            }

            result.final_error = vectorInfinityNorm(diff);
            result.iterations = iteration;

            if (result.final_error < eps) {
                result.converged = true;
                result.x = x_new;
                result.residual_norm = residualInfinityNorm(s, result.x);
                result.message = result.guaranteed_convergence
                    ? "Метод Якоби сошёлся для трёхдиагональной системы."
                    : "Метод Якоби сошёлся для трёхдиагональной системы без
априорной гарантии.";
                return result;
            }

            x_old.swap(x_new);
        }

        result.x = x_old;

```

```

        result.residual_norm = residualInfinityNorm(s, result.x);
        result.message = "Для трёхдиагональной системы достигнут лимит
итераций без достижения требуемой точности.";
        return result;
    }

JacobiResult solveJacobi(const BandedSystem& s, double eps, int
max_iterations) {
    JacobiResult result;

    const int n = static_cast<int>(s.b.size());
    if (n == 0) {
        result.converged = true;
        result.guaranteed_convergence = true;
        result.message = "Пустая система: решение считается найденным.";
        return result;
    }
    if (static_cast<int>(s.diagonals.size()) != s.lower_band +
s.upper_band + 1) {
        throw std::invalid_argument("Некорректное число диагоналей в
ленточной системе.");
    }
    for (const auto& diag : s.diagonals) {
        if (static_cast<int>(diag.size()) != n) {
            throw std::invalid_argument("Все диагонали ленточной системы
должны иметь длину n.");
        }
    }
    if (eps <= 0.0 || max_iterations <= 0) {
        throw std::invalid_argument("eps и max_iterations должны быть
положительными.");
    }

    result.guaranteed_convergence = isStrictlyDiagonallyDominant(s, 1e-
12);

    Vector x_old(n, 0.0);
    Vector x_new(n, 0.0);

    for (int iteration = 1; iteration <= max_iterations; ++iteration) {
        for (int i = 0; i < n; ++i) {
            const double diag = getBandedValue(s, i, i);
            if (absd(diag) <= 1e-12) {
                result.message = "На главной диагонали ленточной матрицы
встретился нулевой элемент.";
                result.iterations = iteration - 1;
                return result;
            }

            double sum = 0.0;
            const int left = std::max(0, i - s.lower_band);
            const int right = std::min(n - 1, i + s.upper_band);
            for (int j = left; j <= right; ++j) {
                if (j == i) {
                    continue;
                }
                sum += getBandedValue(s, i, j) * x_old[j];
            }
        }
    }
}

```

```

        x_new[i] = (s.b[i] - sum) / diag;
    }

    Vector diff(n, 0.0);
    for (int i = 0; i < n; ++i) {
        diff[i] = x_new[i] - x_old[i];
    }

    result.final_error = vectorInfinityNorm(diff);
    result.iterations = iteration;

    if (result.final_error < eps) {
        result.converged = true;
        result.x = x_new;
        result.residual_norm = residualInfinityNorm(s, result.x);
        result.message = result.guaranteed_convergence
            ? "Метод Якоби сошёлся для ленточной системы."
            : "Метод Якоби сошёлся для ленточной системы без строгой
априорной гарантии.";
        return result;
    }

    x_old.swap(x_new);
}

result.x = x_old;
result.residual_norm = residualInfinityNorm(s, result.x);
result.message = "Для ленточной системы достигнут лимит итераций без
достижения требуемой точности.";
return result;
}

Matrix generateHilbertMatrix(int n) {
    if (n <= 0) {
        throw std::invalid_argument("Размер матрицы Гильберта должен быть
положительным.");
    }

    Matrix a(n, Vector(n, 0.0));
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            a[i][j] = 1.0 / static_cast<double>(i + j + 1);
        }
    }
    return a;
}

void generateHilbertSystem(int n, Matrix& a, Vector& b, const Vector&
exact_x) {
    if (static_cast<int>(exact_x.size()) != n) {
        throw std::invalid_argument("Размер exact_x должен совпадать с
размером матрицы Гильберта.");
    }

    a = generateHilbertMatrix(n);
    b = multiply(a, exact_x);
}

```



```

void generateDenseDiagonallyDominantSystem(int n, Matrix& a, Vector& b,
const Vector& exact_x, unsigned seed) {
    if (n <= 0) {
        throw std::invalid_argument("Размер системы должен быть
положительным.");
    }
    if (static_cast<int>(exact_x.size()) != n) {
        throw std::invalid_argument("Размер exact_x должен совпадать с
размером системы.");
    }

    std::mt19937 gen(seed);
    std::uniform_real_distribution<double> dist(-1.0, 1.0);
    std::uniform_real_distribution<double> margin_dist(1.0, 3.0);

    a.assign(n, Vector(n, 0.0));
    b.assign(n, 0.0);

    for (int i = 0; i < n; ++i) {
        double sum_abs = 0.0;
        for (int j = 0; j < n; ++j) {
            if (i == j) {
                continue;
            }
            a[i][j] = dist(gen);
            sum_abs += absd(a[i][j]);
        }
        a[i][i] = sum_abs + margin_dist(gen);
    }

    b = multiply(a, exact_x);
}

TridiagonalSystem generateLargeTridiagonalSystem(int n, const Vector&
exact_x) {
    if (n <= 0) {
        throw std::invalid_argument("Размер системы должен быть
положительным.");
    }
    if (static_cast<int>(exact_x.size()) != n) {
        throw std::invalid_argument("Размер exact_x должен совпадать с
размером системы.");
    }

    TridiagonalSystem s;
    s.lower.assign(std::max(0, n - 1), -1.0);
    s.diag.assign(n, 4.0);
    s.upper.assign(std::max(0, n - 1), -1.0);
    s.b.assign(n, 0.0);

    for (int i = 0; i < n; ++i) {
        s.b[i] = s.diag[i] * exact_x[i];
        if (i > 0) {
            s.b[i] += s.lower[i - 1] * exact_x[i - 1];
        }
        if (i + 1 < n) {
            s.b[i] += s.upper[i] * exact_x[i + 1];
        }
    }
}

```

```

        }
    }

    return s;
}

BandedSystem generatePentadiagonalSystem(int n, const Vector& exact_x,
unsigned seed) {
    return generateBandedDiagonallyDominantSystem(n, 2, 2, exact_x,
seed);
}

BandedSystem generateBandedDiagonallyDominantSystem(
    int n,
    int lower_band,
    int upper_band,
    const Vector& exact_x,
    unsigned seed
) {
    if (n <= 0) {
        throw std::invalid_argument("Размер системы должен быть
положительным.");
    }
    if (static_cast<int>(exact_x.size()) != n) {
        throw std::invalid_argument("Размер exact_x должен совпадать с
размером системы.");
    }

    BandedSystem s = makeEmptyBandedSystem(n, lower_band, upper_band);

    std::mt19937 gen(seed);
    std::uniform_real_distribution<double> dist(-1.0, 1.0);
    std::uniform_real_distribution<double> margin_dist(1.0, 3.0);

    for (int i = 0; i < n; ++i) {
        double sum_abs = 0.0;

        const int left = std::max(0, i - lower_band);
        const int right = std::min(n - 1, i + upper_band);

        for (int j = left; j <= right; ++j) {
            if (i == j) {
                continue;
            }
            const double value = dist(gen);
            setBandedValue(s, i, j, value);
            sum_abs += absd(value);
        }

        setBandedValue(s, i, i, sum_abs + margin_dist(gen));
    }

    s.b = multiply(s, exact_x);
    return s;
}

BandedSystem generateSymmetricPositiveDefiniteBandedSystem(
    int n,

```

```

    int half_band,
    const Vector& exact_x,
    unsigned seed
) {
    if (n <= 0) {
        throw std::invalid_argument("Размер системы должен быть
положительным.");
    }
    if (static_cast<int>(exact_x.size()) != n) {
        throw std::invalid_argument("Размер exact_x должен совпадать с
размером системы.");
    }

    BandedSystem s = makeEmptyBandedSystem(n, half_band, half_band);

    std::mt19937 gen(seed);
    std::uniform_real_distribution<double> weight_dist(0.05, 0.7);
    std::uniform_real_distribution<double> margin_dist(1.0, 2.0);

    // Сначала заполняем симметричные внедиагональные элементы
отрицательными числами.
    for (int i = 0; i < n; ++i) {
        const int right = std::min(n - 1, i + half_band);
        for (int j = i + 1; j <= right; ++j) {
            const double w = weight_dist(gen);
            setBandedValue(s, i, j, -w);
            setBandedValue(s, j, i, -w);
        }
    }

    // Теперь делаем диагональ достаточно большой.
    // Получается симметричная строго диагонально доминирующая матрица,
    // а значит она невырождена и хорошо подходит для метода Якоби.
    for (int i = 0; i < n; ++i) {
        double sum_abs = 0.0;
        const int left = std::max(0, i - half_band);
        const int right = std::min(n - 1, i + half_band);
        for (int j = left; j <= right; ++j) {
            if (i == j) {
                continue;
            }
            sum_abs += absd(getBandedValue(s, i, j));
        }
        setBandedValue(s, i, i, sum_abs + margin_dist(gen));
    }

    s.b = multiply(s, exact_x);
    return s;
}

void saveDenseSystemToFile(const Matrix& a, const Vector& b, const
std::string& filename, double eps, int max_iterations) {
    std::ofstream fout(filename);
    if (!fout) {
        throw std::runtime_error("Не удалось открыть файл для записи: " +
filename);
    }
}

```

```

const int n = static_cast<int>(a.size());
fout << "dense\n";
fout << n << "\n";
fout.setf(std::ios::scientific);
fout.precision(17);

for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
        fout << a[i][j] << (j + 1 == n ? '\n' : ' ');
    }
}
for (int i = 0; i < n; ++i) {
    fout << b[i] << (i + 1 == n ? '\n' : ' ');
}
fout << eps << ' ' << max_iterations << "\n";
}

void saveTridiagonalSystemToFile(const TridiagonalSystem& s, const
std::string& filename, double eps, int max_iterations) {
    std::ofstream fout(filename);
    if (!fout) {
        throw std::runtime_error("Не удалось открыть файл для записи: " +
filename);
    }

    const int n = static_cast<int>(s.diag.size());
    fout << "tridiag\n";
    fout << n << "\n";
    fout.setf(std::ios::scientific);
    fout.precision(17);

    for (int i = 0; i < n - 1; ++i) {
        fout << s.lower[i] << (i + 1 == n - 1 ? '\n' : ' ');
    }
    if (n == 1) {
        fout << "\n";
    }
    for (int i = 0; i < n; ++i) {
        fout << s.diag[i] << (i + 1 == n ? '\n' : ' ');
    }
    for (int i = 0; i < n - 1; ++i) {
        fout << s.upper[i] << (i + 1 == n - 1 ? '\n' : ' ');
    }
    if (n == 1) {
        fout << "\n";
    }
    for (int i = 0; i < n; ++i) {
        fout << s.b[i] << (i + 1 == n ? '\n' : ' ');
    }
    fout << eps << ' ' << max_iterations << "\n";
}

void saveBandedSystemToFile(const BandedSystem& s, const std::string&
filename, double eps, int max_iterations) {
    std::ofstream fout(filename);
    if (!fout) {
        throw std::runtime_error("Не удалось открыть файл для записи: " +
filename);
    }

```

```

    }

    const int n = static_cast<int>(s.b.size());
    fout << "banded\n";
    fout << n << ' ' << s.lower_band << ' ' << s.upper_band << "\n";
    fout.setf(std::ios::scientific);
    fout.precision(17);

    for (const auto& diag : s.diagonals) {
        for (int i = 0; i < n; ++i) {
            fout << diag[i] << (i + 1 == n ? '\n' : ' ');
        }
    }
    for (int i = 0; i < n; ++i) {
        fout << s.b[i] << (i + 1 == n ? '\n' : ' ');
    }
    fout << eps << ' ' << max_iterations << "\n";
}

} // namespace jacobi

```

Название файла: main.cpp

```

#include "jacobi_solver.hpp"

#include <algorithm>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <stdexcept>
#include <string>

using namespace jacobi;

namespace {

void printHelp() {
    std::cout << "Решение СЛУ методом Якоби и генерация тестовых матриц\n\n";
    std::cout << "Режимы работы:\n";
    std::cout << "  1) Интерактивный режим без аргументов\n";
    std::cout << "  2) Генерация системы в файл\n";
    std::cout << "  3) Решение системы из файла\n\n";

    std::cout << "Команды генерации:\n";
    std::cout << "  ./jacobi generate hilbert N OUT [exact_value eps max_iterations]\n";
    std::cout << "  ./jacobi generate dense-dd N OUT [exact_value eps max_iterations seed]\n";
    std::cout << "  ./jacobi generate tridiag N OUT [exact_value eps max_iterations]\n";
    std::cout << "  ./jacobi generate pentadiag N OUT [exact_value eps max_iterations seed]\n";
    std::cout << "  ./jacobi generate banded N LOWER UPPER OUT [exact_value eps max_iterations seed]\n";
    std::cout << "  ./jacobi generate symm-banded N HALF OUT [exact_value eps max_iterations seed]\n\n";
}

```

```

std::cout << "Команда решения:\n";
std::cout << "  ./jacobi solve-file INPUT\n\n";

std::cout << "Пояснения:\n";
std::cout << "  hilbert          - матрица Гильберта (плотная, плохо
обусловленная)\n";
std::cout << "  dense-dd          - плотная строго диагонально
доминирующая матрица\n";
std::cout << "  tridiag          - большая трёхдиагональная матрица\n";
std::cout << "  pentadiag         - большая пятидиагональная матрица\n";
std::cout << "  banded           - общая ленточная диагонально
доминирующая матрица\n";
std::cout << "  symm-banded      - симметричная положительно
определённая ленточная матрица\n\n";

std::cout << "Замечание:\n";
std::cout << "  Для очень больших размеров лучше использовать
structured-форматы\n";
std::cout << "  (tridiag, pentadiag, banded, symm-banded), а не
плотные матрицы.\n";
}

void printDenseAnalysis(const Matrix& a, const Vector& b) {
    const SystemStatus status = analyzeSystem(a, b);

    std::cout << "Анализ системы:\n";
    std::cout << "rank(A)          = " << status.rank_a << "\n";
    std::cout << "rank([A|b]) = " << status.rank_augmented << "\n";

    if (status.type == SolutionType::NONE) {
        std::cout << "Система несовместна, решений нет.\n";
    } else if (status.type == SolutionType::INFINITE) {
        std::cout << "Система имеет бесконечно много решений.\n";
    } else {
        std::cout << "Система имеет единственное решение, матрица
невырождена.\n";
    }
}

void printResult(const JacobiResult& result) {
    std::cout << "\nРезультат метода Якоби:\n";
    std::cout << result.message << "\n";
    std::cout << "Гарантия сходимости: " <<
(result.guaranteed_convergence ? "да" : "нет") << "\n";
    std::cout << "Сошёлся: " << (result.converged ? "да" : "нет") <<
"\n";
    std::cout << "Итераций: " << result.iterations << "\n";
    std::cout << "||x(k+1)-x(k)||_inf = " << result.final_error << "\n";
    std::cout << "||Ax-b||_inf          = " << result.residual_norm <<
"\n";

    if (result.converged) {
        const int preview = static_cast<int>(std::min<std::size_t>(10,
result.x.size()));
        std::cout << "Первые " << preview << " компонент решения:\n";
        for (int i = 0; i < preview; ++i) {
            std::cout << "x[" << i << "] = " << result.x[i] << "\n";
        }
    }
}

```

```

        if (static_cast<int>(result.x.size()) > preview) {
            std::cout << "...\\n";
        }
    }
}

void interactiveMode() {
    std::cout << "Решение СЛУ методом Якоби\\n";
    std::cout << "Введите размер системы n: ";

    int n = 0;
    std::cin >> n;
    if (!std::cin || n <= 0) {
        throw std::runtime_error("Размер системы должен быть
положительным целым числом.");
    }

    Matrix a(n, Vector(n, 0.0));
    Vector b(n, 0.0);

    std::cout << "Введите матрицу A размером " << n << "x" << n << ":\n";
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            std::cin >> a[i][j];
        }
    }

    std::cout << "Введите вектор правой части b из " << n << "
элементов:\n";
    for (int i = 0; i < n; ++i) {
        std::cin >> b[i];
    }

    double eps = 0.0;
    int max_iterations = 0;
    std::cout << "Введите точность eps и максимальное число итераций: ";
    std::cin >> eps >> max_iterations;

    if (!std::cin || eps <= 0.0 || max_iterations <= 0) {
        throw std::runtime_error("eps и число итераций должны быть
положительными.");
    }

    printDenseAnalysis(a, b);
    const SystemStatus status = analyzeSystem(a, b);
    if (status.type != SolutionType::UNIQUE) {
        return;
    }

    const JacobiResult result = solveJacobi(a, b, eps, max_iterations,
true);
    printResult(result);
}

void solveFromFile(const std::string& filename) {
    std::ifstream fin(filename);
    if (!fin) {
        throw std::runtime_error("Не удалось открыть файл: " + filename);
    }
}

```

```

    }

    std::string type;
    fin >> type;
    if (!fin) {
        throw std::runtime_error("Файл пуст или повреждён: " + filename);
    }

    if (type == "dense") {
        int n = 0;
        fin >> n;
        if (!fin || n <= 0) {
            throw std::runtime_error("Некорректный размер плотной матрицы
в файле: " + filename);
        }

        Matrix a(n, Vector(n, 0.0));
        Vector b(n, 0.0);
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j) {
                fin >> a[i][j];
            }
        }
        for (int i = 0; i < n; ++i) {
            fin >> b[i];
        }

        double eps = 0.0;
        int max_iterations = 0;
        fin >> eps >> max_iterations;
        if (!fin) {
            throw std::runtime_error("Не удалось полностью прочитать
плотную систему из файла: " + filename);
        }

        printDenseAnalysis(a, b);
        const SystemStatus status = analyzeSystem(a, b);
        if (status.type != SolutionType::UNIQUE) {
            return;
        }

        const JacobiResult result = solveJacobi(a, b, eps,
max_iterations, true);
        printResult(result);
        return;
    }

    if (type == "tridiag") {
        int n = 0;
        fin >> n;
        if (!fin || n <= 0) {
            throw std::runtime_error("Некорректный размер
трёхдиагональной системы в файле: " + filename);
        }

        TridiagonalSystem s;
        s.lower.assign(std::max(0, n - 1), 0.0);
        s.diag.assign(n, 0.0);
    }

```



```

s.upper.assign(std::max(0, n - 1), 0.0);
s.b.assign(n, 0.0);

for (int i = 0; i < n - 1; ++i) {
    fin >> s.lower[i];
}
for (int i = 0; i < n; ++i) {
    fin >> s.diag[i];
}
for (int i = 0; i < n - 1; ++i) {
    fin >> s.upper[i];
}
for (int i = 0; i < n; ++i) {
    fin >> s.b[i];
}

double eps = 0.0;
int max_iterations = 0;
fin >> eps >> max_iterations;
if (!fin) {
    throw std::runtime_error("Не удалось полностью прочитать
трёхдиагональную систему из файла: " + filename);
}

std::cout << "Анализ системы:\n";
std::cout << "Трёхдиагональная система сгенерирована так, что
матрица строго диагонально доминирует,\n";
std::cout << "следовательно, она невырождена и имеет единственное
решение.\n";

const JacobiResult result = solveJacobi(s, eps, max_iterations);
printResult(result);
return;
}

if (type == "banded") {
    int n = 0;
    int lower = 0;
    int upper = 0;
    fin >> n >> lower >> upper;
    if (!fin || n <= 0 || lower < 0 || upper < 0) {
        throw std::runtime_error("Некорректные параметры ленточной
системы в файле: " + filename);
    }

    BandedSystem s;
    s.lower_band = lower;
    s.upper_band = upper;
    s.diagonals.assign(lower + upper + 1, Vector(n, 0.0));
    s.b.assign(n, 0.0);

    for (auto& diag : s.diagonals) {
        for (int i = 0; i < n; ++i) {
            fin >> diag[i];
        }
    }
    for (int i = 0; i < n; ++i) {
        fin >> s.b[i];
    }
}

```

```

    }

    double eps = 0.0;
    int max_iterations = 0;
    fin >> eps >> max_iterations;
    if (!fin) {
        throw std::runtime_error("Не удалось полностью прочитать
ленточную систему из файла: " + filename);
    }

    std::cout << "Анализ системы:\n";
    std::cout << "Ленточная система была сгенерирована в специальном
формате.\n";
    std::cout << "Для таких генераторов матрица строится строго
диагонально доминирующей,\n";
    std::cout << "поэтому она невырождена и имеет единственное
решение.\n";

    const JacobiResult result = solveJacobi(s, eps, max_iterations);
    printResult(result);
    return;
}

throw std::runtime_error("Неизвестный тип системы в файле: " + type);
}

int parseInt(const std::string& s) {
    return std::stoi(s);
}

double parseDouble(const std::string& s) {
    return std::stod(s);
}

void generateCommand(int argc, char* argv[]) {
    if (argc < 5) {
        throw std::runtime_error("Недостаточно аргументов для generate.
Используй ./jacobi help");
    }

    const std::string kind = argv[2];
    const double default_exact_value = 1.0;
    const double default_eps = 1e-10;
    const int default_max_iterations = 100000;
    const unsigned default_seed = 123456789u;

    if (kind == "hilbert") {
        const int n = parseInt(argv[3]);
        const std::string out = argv[4];
        const double exact_value = argc >= 6 ? parseDouble(argv[5]) :
default_exact_value;
        const double eps = argc >= 7 ? parseDouble(argv[6]) :
default_eps;
        const int max_iterations = argc >= 8 ? parseInt(argv[7]) :
default_max_iterations;

        Vector exact(n, exact_value);
        Matrix a;

```

```

    Vector b;
    generateHilbertSystem(n, a, b, exact);
    saveDenseSystemToFile(a, b, out, eps, max_iterations);

    std::cout << "Сгенерирована матрица Гильберта " << n << "x" << n
                << " и сохранена в файл " << out << "\n";
    return;
}

if (kind == "dense-dd") {
    const int n = parseInt(argv[3]);
    const std::string out = argv[4];
    const double exact_value = argc >= 6 ? parseDouble(argv[5]) :
default_exact_value;
    const double eps = argc >= 7 ? parseDouble(argv[6]) :
default_eps;
    const int max_iterations = argc >= 8 ? parseInt(argv[7]) :
default_max_iterations;
    const unsigned seed = argc >= 9 ?
static_cast<unsigned>(parseInt(argv[8])) : default_seed;

    Vector exact(n, exact_value);
    Matrix a;
    Vector b;
    generateDenseDiagonallyDominantSystem(n, a, b, exact, seed);
    saveDenseSystemToFile(a, b, out, eps, max_iterations);

    std::cout << "Сгенерирована плотная диагонально доминирующая
матрица " << n << "x" << n
                << " и сохранена в файл " << out << "\n";
    return;
}

if (kind == "tridiag") {
    const int n = parseInt(argv[3]);
    const std::string out = argv[4];
    const double exact_value = argc >= 6 ? parseDouble(argv[5]) :
default_exact_value;
    const double eps = argc >= 7 ? parseDouble(argv[6]) :
default_eps;
    const int max_iterations = argc >= 8 ? parseInt(argv[7]) :
default_max_iterations;

    Vector exact(n, exact_value);
    TridiagonalSystem s = generateLargeTridiagonalSystem(n, exact);
    saveTridiagonalSystemToFile(s, out, eps, max_iterations);

    std::cout << "Сгенерирована трёхдиагональная матрица размера " <<
n
                << " и сохранена в файл " << out << "\n";
    return;
}

if (kind == "pentadiag") {
    const int n = parseInt(argv[3]);
    const std::string out = argv[4];
    const double exact_value = argc >= 6 ? parseDouble(argv[5]) :
default_exact_value;

```

```

        const double eps = argc >= 7 ? parseDouble(argv[6]) :
default_eps;
        const int max_iterations = argc >= 8 ? parseInt(argv[7]) :
default_max_iterations;
        const unsigned seed = argc >= 9 ?
static_cast<unsigned>(parseInt(argv[8])) : default_seed;

        Vector exact(n, exact_value);
        BandedSystem s = generatePentadiagonalSystem(n, exact, seed);
        saveBandedSystemToFile(s, out, eps, max_iterations);

        std::cout << "Сгенерирована пятидиагональная матрица размера " <<
n
        << " и сохранена в файл " << out << "\n";
        return;
    }

    if (kind == "banded") {
        if (argc < 7) {
            throw std::runtime_error("Для banded нужно: ./jacobi generate
banded N LOWER UPPER OUT [exact_value eps max_iterations seed]");
        }

        const int n = parseInt(argv[3]);
        const int lower = parseInt(argv[4]);
        const int upper = parseInt(argv[5]);
        const std::string out = argv[6];
        const double exact_value = argc >= 8 ? parseDouble(argv[7]) :
default_exact_value;
        const double eps = argc >= 9 ? parseDouble(argv[8]) :
default_eps;
        const int max_iterations = argc >= 10 ? parseInt(argv[9]) :
default_max_iterations;
        const unsigned seed = argc >= 11 ?
static_cast<unsigned>(parseInt(argv[10])) : default_seed;

        Vector exact(n, exact_value);
        BandedSystem s = generateBandedDiagonallyDominantSystem(n, lower,
upper, exact, seed);
        saveBandedSystemToFile(s, out, eps, max_iterations);

        std::cout << "Сгенерирована ленточная матрица размера " << n
        << " с нижней лентой " << lower
        << " и верхней лентой " << upper
        << ", файл: " << out << "\n";
        return;
    }

    if (kind == "symm-banded") {
        if (argc < 6) {
            throw std::runtime_error("Для symm-banded нужно: ./jacobi
generate symm-banded N HALF OUT [exact_value eps max_iterations seed]");
        }

        const int n = parseInt(argv[3]);
        const int half = parseInt(argv[4]);
        const std::string out = argv[5];

```

```

        const double exact_value = argc >= 7 ? parseDouble(argv[6]) :
default_exact_value;
        const double eps = argc >= 8 ? parseDouble(argv[7]) :
default_eps;
        const int max_iterations = argc >= 9 ? parseInt(argv[8]) :
default_max_iterations;
        const unsigned seed = argc >= 10 ?
static_cast<unsigned>(parseInt(argv[9])) : default_seed;

        Vector exact(n, exact_value);
        BandedSystem s = generateSymmetricPositiveDefiniteBandedSystem(n,
half, exact, seed);
        saveBandedSystemToFile(s, out, eps, max_iterations);

        std::cout << "Сгенерирована симметричная положительно
определённая ленточная матрица размера " << n
                << " с полушириной ленты " << half
                << ", файл: " << out << "\n";
        return;
    }

    throw std::runtime_error("Неизвестный вид генератора: " + kind);
}

} // namespace

int main(int argc, char* argv[]) {
    std::cout << std::fixed << std::setprecision(12);

    try {
        if (argc == 1) {
            interactiveMode();
            return 0;
        }

        const std::string command = argv[1];
        if (command == "help" || command == "--help" || command == "-h")
{
            printHelp();
            return 0;
        }
        if (command == "generate") {
            generateCommand(argc, argv);
            return 0;
        }
        if (command == "solve-file") {
            if (argc != 3) {
                throw std::runtime_error("Использование: ./jacobi solve-
file INPUT");
            }
            solveFromFile(argv[2]);
            return 0;
        }

        throw std::runtime_error("Неизвестная команда. Используйте ./jacobi
help");
    } catch (const std::exception& ex) {
        std::cerr << "Ошибка: " << ex.what() << "\n";
    }
}

```

```

        return 1;
    }
}

```

Название файла: test.cpp

```

#include "jacobi_solver.hpp"

#include <cassert>
#include <chrono>
#include <cmath>
#include <iomanip>
#include <iostream>
#include <string>

using namespace jacobi;

namespace {

void printHeader(const std::string& title) {
    std::cout << "\n=====\\n";
    std::cout << title << "\\n";
    std::cout << "=====\\n";
}

void assertClose(double actual, double expected, double eps, const
std::string& message) {
    if (std::fabs(actual - expected) > eps) {
        std::cerr << "Провал проверки: " << message
            << ", actual = " << actual
            << ", expected = " << expected << "\\n";
        std::abort();
    }
}

void assertVectorClose(const Vector& actual, const Vector& expected,
double eps, const std::string& message) {
    if (actual.size() != expected.size()) {
        std::cerr << "Провал проверки: размеры векторов различаются. " <<
message << "\\n";
        std::abort();
    }
    for (std::size_t i = 0; i < actual.size(); ++i) {
        if (std::fabs(actual[i] - expected[i]) > eps) {
            std::cerr << "Провал проверки: " << message
                << ", index = " << i
                << ", actual = " << actual[i]
                << ", expected = " << expected[i] << "\\n";
            std::abort();
        }
    }
}

void testBasicCase() {
    printHeader("Тест 1. Базовый диагонально доминирующий случай");

    Matrix a = {
        {10.0, -1.0, 2.0},
        {-1.0, 11.0, -1.0},
    }
}

```

```

        { 2.0, -1.0, 10.0}
    };
    Vector exact = {1.0, 2.0, -1.0};
    Vector b = {6.0, 22.0, -10.0};

    const auto start = std::chrono::steady_clock::now();
    const JacobiResult result = solveJacobi(a, b, 1e-12, 10000, true);
    const auto finish = std::chrono::steady_clock::now();

    assert(result.converged);
    assert(result.guaranteed_convergence);
    assertVectorClose(result.x, exact, 1e-8, "Базовый тест: неверное
решение");

    const          auto          ms          =
std::chrono::duration_cast<std::chrono::milliseconds>(finish
start).count();
    std::cout << "OK, итераций = " << result.iterations
        << ", невязка = " << result.residual_norm
        << ", время = " << ms << " мс\n";
}

void testNoSolution() {
    printHeader("Тест 2. Несовместная система");

    Matrix a = {
        {1.0, 1.0},
        {2.0, 2.0}
    };
    Vector b = {2.0, 5.0};

    const SystemStatus status = analyzeSystem(a, b);
    assert(status.type == SolutionType::NONE);

    const JacobiResult result = solveJacobi(a, b);
    assert(!result.converged);

    std::cout << "OK, программа правильно определила отсутствие
решения.\n";
}

void testInfiniteSolutions() {
    printHeader("Тест 3. Система с бесконечно многими решениями");

    Matrix a = {
        {1.0, 1.0},
        {2.0, 2.0}
    };
    Vector b = {2.0, 4.0};

    const SystemStatus status = analyzeSystem(a, b);
    assert(status.type == SolutionType::INFINITE);

    const JacobiResult result = solveJacobi(a, b);
    assert(!result.converged);

    std::cout << "OK, программа правильно определила бесконечное число
решений.\n";
}

```

```

}

void testHilbert2() {
    printHeader("Тест 4. Матрица Гильберта 2x2");

    Matrix a;
    Vector b;
    Vector exact = {1.0, 1.0};
    generateHilbertSystem(2, a, b, exact);

    const JacobiResult result = solveJacobi(a, b, 1e-12, 100000, true);

    assert(result.converged);
    assertVectorClose(result.x, exact, 1e-6, "Матрица Гильберта 2x2: неверное решение");
    std::cout << "ОК, для H2 метод Якоби сошёлся.\n";
}

void testHilbert3() {
    printHeader("Тест 5. Матрица Гильберта 3x3");

    Matrix a;
    Vector b;
    Vector exact = {1.0, 1.0, 1.0};
    generateHilbertSystem(3, a, b, exact);

    const JacobiResult result = solveJacobi(a, b, 1e-12, 500, true);

    assert(!result.guaranteed_convergence);
    std::cout << "Сообщение программы: " << result.message << "\n";
    std::cout << "ОК, программа корректно показала, что гарантии сходимости нет.\n";
}

void testDenseDD() {
    printHeader("Тест 6. Плотная диагонально доминирующая матрица 150x150");

    const int n = 150;
    Matrix a;
    Vector b;
    Vector exact(n, 1.0);
    generateDenseDiagonallyDominantSystem(n, a, b, exact, 42u);

    const auto start = std::chrono::steady_clock::now();
    const JacobiResult result = solveJacobi(a, b, 1e-10, 10000, true);
    const auto finish = std::chrono::steady_clock::now();

    assert(result.converged);
    assert(result.guaranteed_convergence);
    assertClose(result.residual_norm, 0.0, 1e-6, "Плотная диагонально доминирующая система: большая невязка");

    const auto ms =
std::chrono::duration_cast<std::chrono::milliseconds>(finish
start).count();
    std::cout << "ОК, итераций = " << result.iterations
        << ", невязка = " << result.residual_norm

```



```

        << ", время = " << ms << " мс\n";
    }

void testLargeTridiag1000() {
    printHeader("Тест 7. Большая система 1000x1000 (трёхдиагональная)");

    const int n = 1000;
    Vector exact(n, 1.0);
    TridiagonalSystem s = generateLargeTridiagonalSystem(n, exact);

    const auto start = std::chrono::steady_clock::now();
    const JacobiResult result = solveJacobi(s, 1e-10, 100000);
    const auto finish = std::chrono::steady_clock::now();

    assert(result.converged);
    assertClose(result.residual_norm, 0.0, 1e-6, "Большая трёхдиагональная
система: большая невязка");

    const auto ms =
std::chrono::duration_cast<std::chrono::milliseconds>(finish
start).count();
    std::cout << "ОК, итераций = " << result.iterations
        << ", невязка = " << result.residual_norm
        << ", время = " << ms << " мс\n";
}

void testLargePentadiag2000() {
    printHeader("Тест 8. Большая система 2000x2000 (пятидиагональная)");

    const int n = 2000;
    Vector exact(n, 1.0);
    BandedSystem s = generatePentadiagonalSystem(n, exact, 7u);

    const auto start = std::chrono::steady_clock::now();
    const JacobiResult result = solveJacobi(s, 1e-10, 100000);
    const auto finish = std::chrono::steady_clock::now();

    assert(result.converged);
    assert(result.guaranteed_convergence);
    assertClose(result.residual_norm, 0.0, 1e-6, "Пятидиагональная
система: большая невязка");

    const auto ms =
std::chrono::duration_cast<std::chrono::milliseconds>(finish
start).count();
    std::cout << "ОК, итераций = " << result.iterations
        << ", невязка = " << result.residual_norm
        << ", время = " << ms << " мс\n";
}

void testLargeBanded3000() {
    printHeader("Тест 9. Большая система 3000x3000 (ленточная, ширина 3
снизу и 4 сверху)");

    const int n = 3000;
    Vector exact(n, 1.0);
    BandedSystem s = generateBandedDiagonallyDominantSystem(n, 3, 4,
exact, 11u);

```

```

const auto start = std::chrono::steady_clock::now();
const JacobiResult result = solveJacobi(s, 1e-10, 100000);
const auto finish = std::chrono::steady_clock::now();

assert(result.converged);
assert(result.guaranteed_convergence);
assertClose(result.residual_norm, 0.0, 1e-6, "Ленточная система:
большая невязка");

const auto ms =
std::chrono::duration_cast<std::chrono::milliseconds>(finish
start).count();
std::cout << "ОК, итераций = " << result.iterations
<< ", невязка = " << result.residual_norm
<< ", время = " << ms << " мс\n";
}

void testLargeSymmBanded10000() {
    printHeader("Тест 10. Большая система 10000x10000 (симметричная SPD-
ленточная)");

    const int n = 10000;
    Vector exact(n, 1.0);
    BandedSystem s = generateSymmetricPositiveDefiniteBandedSystem(n, 3,
exact, 13u);

    const auto start = std::chrono::steady_clock::now();
    const JacobiResult result = solveJacobi(s, 1e-8, 100000);
    const auto finish = std::chrono::steady_clock::now();

    assert(result.converged);
    assert(result.guaranteed_convergence);
    assertClose(result.residual_norm, 0.0, 1e-5, "Симметричная SPD-
ленточная система: большая невязка");

    const auto ms =
std::chrono::duration_cast<std::chrono::milliseconds>(finish
start).count();
    std::cout << "ОК, итераций = " << result.iterations
<< ", невязка = " << result.residual_norm
<< ", время = " << ms << " мс\n";
}

} // namespace

int main() {
    std::cout << std::fixed << std::setprecision(10);

    testBasicCase();
    testNoSolution();
    testInfiniteSolutions();
    testHilbert2();
    testHilbert3();
    testDenseDD();
    testLargeTridiag1000();
    testLargePentadiag2000();
    testLargeBanded3000();

```

```
testLargeSymmBanded10000();  
  
std::cout << "\nВсе тесты успешно пройдены.\n";  
return 0;  
}
```

ПРИЛОЖЕНИЕ В

ТЕСТИРОВАНИЕ

Таблица тестирования задачи:

№	Ввод (с описанием)	Вывод (включая решение)	Время	Память
1	Базовый диагонально-доминирующий случай: $n=3$, $A = \begin{bmatrix} 10 & -1 & 2 \\ 1 & 11 & -1 \\ 2 & -1 & 10 \end{bmatrix}$, $x_{\text{true}} = [1, 2, -1]$, $b = [6, 22, -10]$	Единственное решение, метод сошёлся, $x \approx [1, 2, -1]$, iterations=22, residual_norm ≈ 0	0 мс	96 Б
2	Несовместная система: $n=2$, $A = \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix}$, $b = [2, 5]$	Решений нет, система несовместна, метод не запускается до получения решения, x отсутствует	0 мс	48 Б
3	Система с бесконечно многими решениями: $n=2$, $A = \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix}$, $b = [2, 4]$	Бесконечно много решений, единственное решение отсутствует, метод не выдаёт x	0 мс	48 Б
4	Матрица Гильберта 2×2 : $H(i,j) = 1/(i+j+1)$, $x_{\text{true}} = [1, 1]$, $b = H * x_{\text{true}}$	Метод сошёлся, $x \approx [1, 1]$, решение найдено корректно	0 мс	48 Б

5	Матрица Гильберта 3x3: $H(i,j)=1/(i+j+1)$, $x_{true}=[1,1,1]$, $b=H*x_{true}$	Гарантии сходимости нет; при лимите 500 итераций требуемая точность не достигнута, решение не принято	0 мс	96 Б
6	Плотная диагонально- доминирующая матрица 150x150, $x_{true}=[1,1,...,1]$	Единственное решение, метод сошёлся, $x \approx [1,1,...,1]$, iterations=12, residual_norm $\approx 1e-10$	3 мс	181200 Б
7	Большая система 1000x1000: трёхдиагональная, diag=4, lower=upper=-1, $x_{true}=ones$	Единственное решение, метод сошёлся, $x \approx [1,1,...,1]$, iterations=34, residual_norm $\approx 1e-10$	0 мс	31984 Б
8	Большая система 2000x2000: пятидиагональная ленточная, $x_{true}=ones$	Единственное решение, метод сошёлся, $x \approx [1,1,...,1]$, iterations=29, residual_norm $\approx 1e-10$	0 мс	96000 Б
9	Большая система 3000x3000: ленточная, 3 диагонали снизу и 4 сверху, $x_{true}=ones$	Единственное решение, метод сошёлся, $x \approx [1,1,...,1]$, iterations=28, residual_norm $\approx 1e-10$	1 мс	216000 Б

10	Большая система 10000x10000: симметричная SPD-ленточная, half_band=3, x_true=ones	Единственное решение, метод сошёлся, $x \approx [1, 1, \dots, 1]$, iterations=43, residual_norm $\approx 2.66e-8$	7 мс	640000 Б
----	--	--	------	----------

Таблица 1 – Тестирование алгоритма.